Simulated Annealing for finding Agreement Forests

Manuel Ronhaar ✉
Utrecht University, Netherlands

Abstract

To participate in the PACE 2026 heuristic challenge we implement a simulated annealing algorithm for the Maximum-Agreement Forest. To prove that this tightly approximates the optimal solution and be eligible for the lower bound track we also implement column generation. Our final submission is an integration of both techniques.

2012 ACM Subject Classification Theory of computation → Random search heuristics

Keywords and phrases Simulated Annealing, Column Generation, Phylogenetics, PACE challenge

1 Introduction

The Maximum-Agreement Forest (MAF) problem arises in phylogenetics, the study of evolutionary history, when comparing the heritage of different genes. We state it as: Given multiple bijectively labelled trees, return a minimum sized forest that can be obtained from any of the input trees by removing edges. It was proven NP-hard by Bordewich and Semple[1].

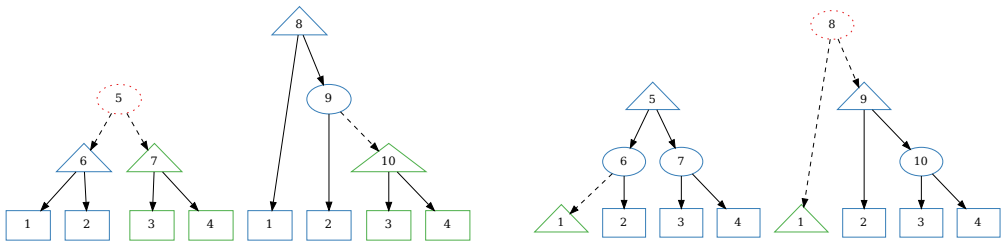
This year's Parameterized Algorithms and Computational Experiments (PACE) challenge is centred around the rooted MAF problem, providing three tracks for competition. One track asks to heuristically find a good solution, which to the best of our knowledge does not have previous research aiming specifically at this goal. The lower bound track requires a solution that is within some bound of optimality. Approximation algorithms exist, but these give a larger approximation factor than allowed for most instances. A promising technique for this track is column generation on the trees of the MAF, which seems to give a tight linear approximation as shown recently by Frohn, Kelk and Vychytilova [2]. The final track asks for an optimal solution, which we do not attempt in this submission.

The first and main contribution of this paper is a simulated annealing technique for quickly finding a good agreement forest as described in section 2. To get a tight lower bound we also implement column generation as specified in section 3. Our final submission combines both methods, where simulated annealing is used as an initial solution providing initial columns, which are then improved by column generation which also provides a lower bound. The last minute of run time is given to an ILP solver to construct the best solution based on the combined column pool. More details on the submission are given in section 4.

2 Simulated Annealing

Simulated annealing is a local search method in which small modifications to a solution are iteratively evaluated. Changes that improve solution quality are accepted, while deteriorating moves may also be accepted with a certain probability depending on their negative impact. Occasionally allowing moves that deteriorate the solution enables the search process to escape local optima. Simulated annealing was first introduced by Kirkpatrick et al. in 1983 [3].

In our simulated annealing the state space is any valid agreement forest, starting at the forest where every label has its own tree. The neighbourhood (in essence the modification we consider) consists of connecting two leaves in the agreement forest and cutting all edges that are no longer valid because of the new connection. In the rest of this section we first go in more detail on this neighbourhood and then describe the acceptance criterion.



(a) Forest before neighbourhood application **(b)** After connecting leaves 2 and 3

Figure 1 Visualisation of how the solution forest would change when connecting leaves 2 and 3.

To illustrate the neighbourhood function, we first give an example in figure 1. The two input trees are ((1,2),(3,4)) and (1,((3,4),2)) and the current state has solution trees (1,2) and (3,4). When we evaluate the neighbourhood connecting leaves 2 and 3, we find the solution tree (1,2) needs to be cut for the solution forest to remain valid while (3,4) can remain connected.

To evaluate which edges need to be cut we draw the path from one leaf to the other in both trees. If the path goes over a solution tree that contains neither the first nor the second leaf, that edge is in conflict with the new connection and needs to be cut. The edges of the solution trees, that do contain one of the chosen leaves are cut if they are encountered as a branch on one of the paths, but not on the other. In the example tree, the edge 1 to (1,2) is encountered as branch in the left tree, but not in the right tree.

To effectively calculate which edges need to be cut, for all nodes in both input trees we persistently keep track of which solution tree nodes they correspond to. When evaluating a move, the cuts are not directly made but only the required number is calculated. This enables about a million evaluations per second on a modern CPU.

An interesting question is how to choose the leaves to connect. The first implementation drew two leaves independently and uniformly at random. Especially in the larger instances this resulted in some good connections never being evaluated. So instead we opt for a selection biased towards close together leaves. At every iteration we choose one of the input trees and in that input tree a non-leaf node with its children not both in the same solution tree. From the children of this chosen node we randomly go left or right until we come across a leaf. Additionally, as soon as we enter a node in a solution tree, we never leave this solution tree.

If a neighbourhood move decreases the number of trees or keeps it equal, the move is always accepted. Otherwise the probability of acceptance is:

$$P = e^{\triangle S/T}$$

where $\triangle S$ is the negative increase in number of trees and T the temperature. The temperature starts at an initial value of 1 and decreases geometrically to 0.01. This means at the start, a move increasing the number of trees by one is accepted with probability one third, while near the end such a move is almost certainly rejected ($P < 10^{-40}$) and only moves that do not deteriorate the solution are accepted.

3 Column Generation

3.1 Reduced Master Problem

In our relaxed master problem (RMP) the columns are trees of an agreement forest. Any column has as properties the number of connections it makes (number of leaves - 1) and for any interior node whether it is used. The optimization objective is to maximize the number of connections without using an interior node more than once. Of course, no tree can be included a negative number of times.

$$\begin{array}{lll} \max & \sum_{i \in I} x_i c_i & \\ \text{s. t.} & \sum_{i \in I} x_i d_{in} \leq 1 & \forall_{n \in N} \\ & x_i \geq 0 & \forall_{i \in I} \end{array}$$

Maximizing the number of connections is equal to minimizing the number of trees in the agreement forest, as the number of trees is the number of leaves minus the number of total connections.

3.1.1 Stabilizing Shadow costs

To stabilize the shadow costs between column generation iterations, we chose to apply L1 penalisation. Stabilisation of the shadow costs ensures the input to the pricing problem changes less between iterations, meaning less columns will have reduced cost. To apply L1 penalisation, we need to translate the RMP to its dual form, as here shadow costs are variables that can be manipulated. The dual form of the original RMP problem is as follows:

$$\begin{array}{lll} \min & \sum_{n \in N} y_n & \\ \text{s. t.} & \sum_{n \in N} d_{in} y_n \geq c_i & \forall_{i \in I} \\ & y_n \geq 0 & \forall_{n \in N} \end{array}$$

We add a constraint that the new shadow price y_n is equal to the previous shadow costs $\bar{y}_n$, except for positive and negative deviations a_n and b_n, which we penalize with an insignificant weight ϵ in the objective. The new dual then looks as follows:

$$\begin{array}{lll} \min & \sum_{n \in N} y_n + \epsilon \cdot \sum_{n \in N} (a_n + b_n) & \\ \text{s. t.} & \sum_{n \in N} d_{in} y_n \geq c_i & \forall_{i \in I} \\ & y_n + a_n - b_n = \bar{y}_n & \forall_{n \in N} \\ & y_n, a_n, b_n \geq 0 & \forall_{n \in N} \end{array}$$

Taking the dual of the dual, we get an updated RMP:

$$\begin{array}{lll} \max & \sum_{i \in I} x_i c_i + \bar{y}_n \sum_{n \in N} \mu_n & \\ \text{s. t.} & \sum_{i \in I} x_i d_{in} + \mu_n \leq 1 & \forall_{n \in N} \\ & -\epsilon \leq \mu_n \leq \epsilon & \forall_{n \in N} \\ & x_i \geq 0 & \forall_{i \in I} \end{array}$$

Notice that a solution to the original problem is also a solution to this updated relaxed master problem. Therefore, any bound we get is still valid also for the original problem. In the first column generation iteration we do not have previous shadow costs $\bar{y}_n$, instead we use a constant to encourage even spreading of the shadow costs over the different internal nodes. The constant is chosen as 0.25, approximately the average shadow cost of internal nodes in our experimentation.

3.2 Pricing Problem

3.2.1 Idea

To construct new columns we use dynamic programming. We want to maximize the reduced cost c the number of connections minus the shadow prices of the interior nodes used. This can be exactly solved using dynamic programming. The definition of our table is $DP(a, b) =$ maximal c of a tree with a as root in the first tree and b as root in the second tree. We allow the root to have just one child, which is necessary as a building block for other solutions, but never used as a solution.

Initially we fill the DP array with 0 c for $DP(x, x)$ if x is a leaf and $-\infty$ otherwise.

Any DP entry can be formed in a number of ways. If it has two children in the solution the right child of a can be matched with the right child of b and left with left (option 1), or the children can be crossed (option 2). In either case the number of connections is increased by one and decreased by the shadow prices of a and b indicated by π. Subscript r and l indicate the right or left child respectively.

$$\begin{aligned}\text{option}_1 &= 1 - \pi_a - \pi_b + DP(a_r, b_r) + DP(a_l, b_l)\\ \text{option}_2 &= 1 - \pi_a - \pi_b + DP(a_r, b_l) + DP(a_l, b_r)\end{aligned}$$

If a or b has only one child we get four extra options, every time detracting the shadow price of the newly included root. Note that both a and b could have only one child in which case multiple iterations are needed.

$$\begin{aligned}\text{option}_3 &= DP(a, b_r) - \pi_a\\ \text{option}_4 &= DP(a, b_l) - \pi_a\\ \text{option}_5 &= DP(a_r, b) - \pi_b\\ \text{option}_6 &= DP(a_l, b) - \pi_b\end{aligned}$$

For a and b both interior nodes, $DP(a, b)$ is then the best reduced cost of the six options. If one of the nodes is a leaf only the options that do not include children of that node are considered.

During any execution of the pricing problem we add the best tree for any root a in first tree and any root b in the second tree on the condition $DP(a, b) > 0$ and that the entry was created by a join operation (option 1 or option 2).

3.2.2 Implementation

The idea above gives all the results we need, but it doesn't scale well to multiple trees with a table size and running time of $O(n^t)$. Note that most of the entries in the table are obviously not helpful. For example, DP(1, 2) with both 1 and 2 leaves will clearly have a value of $-\infty$, as no tree can have leaf 1 as root in tree 1 and leaf 2 as root in tree 2.

In the implementation we therefore do not use a dynamic programming table of this large size, but instead use lazy evaluation. We use a priority queue to keep track of new entries, and empty this in a lexicographic order of the roots, with each individual root sorted in reverse DFS order. The options to propagate are then:

$$\begin{aligned}DP(a, b_p) &= DP(a, b) - \pi_{b_p}\\ DP(a_p, b) &= DP(a, b) - \pi_{a_p}\\ DP(a_p, b_p) &= DP(a, b) + DP(a_s, b_s) - \pi_{a_p} - \pi_{b_p} + 1\end{aligned}$$

Where the subscript p denotes a parent of the variable and the subscript s denotes the sibling. The assignment only goes on if the new value is better than the previous value, which is kept track of in a hash map. If the assignment does go through, the new entry is also added to the priority queue.

3.2.3 Limiting New Columns

At the start of column generation, a lot of internal nodes have shadow costs lower than their eventual shadow cost. Because of this, the pricing problem will generate columns that have reduced cost with the current shadow costs, but are not useful in the future. To prevent generating too many of these columns we want to limit the number of columns generated. The most obvious way is to only add the columns with the largest reduced cost, but this incentivises big trees, as they include many nodes.

Instead, we choose to increase all shadow costs by some value x as they are used by the pricing problem. This way only the columns which have big reduced cost per internal node are added, reducing the overall number of columns generated. To get a stable number of new columns we found it best to decrease x geometrically. In the final implementation x goes from 0.1 to 0.001 during the first twenty iterations. After this and for small trees with less than 1000 leaves, this measure is dropped completely so all columns with reduced cost are generated and the lower bound guarantees hold.

3.3 Lower Bound Guarantees

Firstly, if all possible solution trees were added to the RMP, the objective would be a valid lower bound for the MAF problem, as the linear relaxation can only give better results than the original integral problem.

Secondly, if no possible columns have reduced cost, we can still conclude we found the optimal linear relaxation objective and thus still have a valid lower bound.

Finally, if columns with negative reduced costs exist, we can still calculate a pricing bound. Namely, the RMP objective can never decrease by more than the sum of the reduced cost of all columns generated by the pricing problem:

$$\sum_{a \in I_{tree1}} \sum_{b \in I_{tree2}} \min(0, c_{a,b}^{min})$$

where I_{tree1} and I_{tree2} are the sets of internal nodes of tree 1 and tree 2 respectively and $c_{a,b}^{min}$ is the minimum reduced cost of a solution tree with node a as root in one tree and b as root in the other tree. To see this holds one could increase the shadow cost of a by $c_{a,b}^{min}$ for all a and b. With this no more negative reduced columns exist and we know a bound by the weak duality theorem.

4 Submission

Our plan was to rely on the simulated annealing for a solution in both the heuristic and lower bound track, and only use column generation to prove a lower bound. However, we found the improvement of the simulated annealing stagnated when it reached almost optimality and column generation was surprisingly effective, also in generating columns to form a solution. That's why we decided to more tightly integrate the two approaches into one procedure used for both tracks.

First, we run simulated annealing on the input trees. For the heuristic track and lower bound instances that require an almost exact solution (with a below 1.005 and b below 2) we set an iteration limit of 50 million, which takes approximately 1 minute. For the other lower bound instances we set as iteration limit the number of leaves squared divided by 5, so small instances can be solved very quickly. The simulated annealing serves as a possible solution and a source of initial columns. Every solution tree and its full subtrees from a certain root are included as columns.

Using these initial columns we start column generation. When the simulated annealing solution is proven sufficient (within bounds for lower bound track/optimal for heuristic) we can early exit this solution. Otherwise we continue until no columns remain with reduced cost or the time limit for column generation elapses (200 seconds for heuristic track and 540 for the lower bound track).

In the heuristic track, we finish off by running an ILP minimizing the number of trees. For the lower bound track we also run an ILP, but check for feasibility within the known bound. If no satisfiable solution exists with the given columns we instead use simulated annealing again with increasingly longer runtimes.

5 Implementation

All algorithms were implemented in C++, using the Google OR-Tools library[1] for exact solvers. As linear programming solver for column generation we use GLOP[2] by Google and as ILP solver we use CBC[3] by the COIN-OR foundation.

References

1 Magnus Bordewich and Charles Semple. On the computational complexity of the rooted subtree prune and regraft distance. *Annals of combinatorics*, 8(4):409–423, 2005. doi:10.1007/s00026-004-0229-z.

2 Martin Frohn, Steven Kelk, and Simona Vychytilova. A branch-&-price approach to the unrooted maximum agreement forest problem. *Operations Research Letters*, page 107364, 2025. doi:10.1016/j.orl.2025.107364.

3 Scott Kirkpatrick, C. Daniel Gelatt Jr, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi:10.1126/science.220.4598.671.

[1] https://github.com/google/or-tools
[2] https://github.com/google/or-tools
[3] https://github.com/coin-or/Cbc